

Serverless Deployment for Large Language Models: Reducing Latency, Cost, and GPU Idle Time

A Technical Report submitted to the Department of Computer Science

Presented to the Faculty of the School of Engineering and Applied Science

University of Virginia • Charlottesville, Virginia

In Partial Fulfillment of the Requirements for the Degree

Bachelor of Science, School of Engineering

Jae-Hyuk Chang

Spring, 2026

Technical Project Team Members

Shutian Luo

On my honor as a University Student, I have neither given nor received unauthorized aid on this assignment as defined by the Honor Guidelines for Thesis-Related Assignments

Haiying Shen, Department of Computer Science

Serverless Deployment for Large Language Models: Reducing Latency, Cost, and GPU Idle Time

CS4980 Capstone Report, 2026

Jae-Hyuk Chang
Computer Science
The University of Virginia
School of Engineering and Applied Science
Charlottesville, Virginia USA
ugw2ad@virginia.edu

Abstract

Large language models (LLMs) are typically deployed using always-on GPU infrastructure, which leads to inefficient resource utilization under dynamic workloads. This project investigates a serverless approach to LLM inference that dynamically provisions GPU resources based on demand.

I implement and evaluate a prototype system that incorporates local caching and scheduling optimizations to reduce cold-start latency. The system is compared against always-on and naive serverless baselines under steady and bursty workloads.

Experimental results show that the serverless approach significantly reduces idle GPU time and operational cost, while maintaining acceptable latency performance. These results demonstrate the potential of serverless inference for efficient large-scale LLM deployment.

1 Introduction

Large language models (LLMs) have emerged as a central component of modern AI, used across a wide range of applications such as dialogue systems, code generation, and content creation. As these models continue to scale, their memory and computational requirements increase significantly, making efficient deployment more challenging.

Modern inference systems typically rely on always-on GPU deployments, where models remain loaded in memory even when there are no incoming requests. While this approach provides low latency, it is inefficient in terms of resource utilization due to the highly dynamic nature of real-world traffic. In practice, workloads often exhibit bursts of requests followed by relatively long idle periods. During these idle times, GPUs remain underutilized while still consuming energy, leading to unnecessary cost.

Recent efforts have explored serverless deployment of neural networks to improve resource efficiency by provisioning compute resources only when needed. However, applying serverless approaches to large language models introduces new challenges, including cold-start overhead and increased latency.

This work presents the design and evaluation of a serverless LLM inference system. The system incorporates techniques such as local model caching and request-aware scheduling to reduce GPU idle time while maintaining acceptable performance.

I evaluate the proposed approach against a traditional always-on deployment and a naive serverless baseline under various workload conditions. The results show that serverless inference can significantly improve resource efficiency and reduce operational cost, while maintaining competitive latency performance.

2 Related Work

Some existing studies have examined serverless and efficient approaches to machine learning inference, with particular attention to LLMs. Among the most relevant works, Fu et al. [2] presented a serverless inference framework for LLMs called ServerlessLLM. The authors applied techniques such as multi-tier checkpoint loading, live migration, and locality-aware scheduling to address issues related to long start-up time. Since it specifically targets low-latency serverless inference for LLMs, this work is the most closely related to the current research.

In addition to LLM-oriented frameworks, prior studies have explored serverless and GPU-based model serving more broadly. Cai et al. [1] developed a stateful model serving system capable of running machine learning models using serverless computing while sharing GPU resources across models. Although the study does not focus specifically on LLMs, it provides useful insights into designing serverless systems for inference tasks that utilize GPU resources.

The literature also includes studies on inference performance optimization and scheduling policies. In this context, Kim et al. [3] presented Oltunes, a system that enables automatic configuration and adaptation of deep learning inference in heterogeneous GPU clusters using online learning methods. Their findings highlight the importance of scheduling and system configuration in improving performance.

Another group of studies focuses on general serverless computing, particularly scheduling policies and start-up overhead. For example, Wang et al. [5] studied resource allocation strategies for serverless systems and analyzed their impact on start-up latency. Similarly, Li et al. [4] investigated container runtime design for high-density serverless deployments, with an emphasis on minimizing start-up latency and supporting high concurrency.

Overall, prior work has made significant progress in serverless serving, scheduling, and resource optimization. However, relatively few studies focus on the trade-offs between latency, GPU idle time, and cost in LLM deployments. This study builds on these ideas and

evaluates a serverless inference system for LLMs under various workloads. Unlike prior work that focuses on complex system-level optimizations, this study emphasizes the effectiveness of simpler strategies, such as keep-warm mechanisms, in reducing cold-start latency in practical settings.

3 System Design

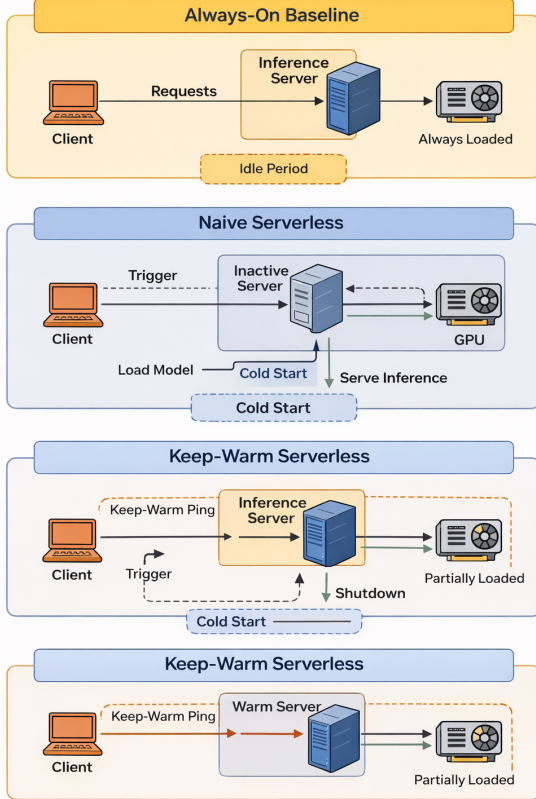


Figure 1: Comparison of always-on, naive serverless, and keep-warm serverless deployment strategies

Each configuration differs in how GPU resources are allocated and managed under dynamic workloads.

3.1 Always-On Baseline

In the always-on baseline, the inference server remains active at all times, and the model is continuously loaded in GPU memory. This design enables low-latency responses since no initialization is required for incoming requests. However, the GPU remains allocated even during idle periods, leading to inefficient resource utilization and unnecessary power consumption.

3.2 Naive Serverless

The naive serverless approach activates the system only when a request arrives. In this configuration, the inference server starts

from an inactive state, loads the model into GPU memory, and then processes the request. After a period of inactivity, the system shuts down. While this approach reduces idle resource usage, it introduces significant cold-start latency due to model loading and initialization overhead.

3.3 Keep-Warm Serverless

The keep-warm serverless approach mitigates cold-start latency by periodically sending lightweight requests to keep the system active. This prevents the model from being fully unloaded and reduces initialization overhead for incoming requests. As a result, this approach achieves a balance between latency and resource efficiency, at the cost of slightly increased resource usage compared to the naive serverless approach.

4 Results

All experiments were conducted under identical hardware conditions using an NVIDIA L4 GPU. Requests were generated using a fixed prompt, and throughput was measured under controlled concurrency levels.

4.1 Baseline (Always-on Deployment)

The baseline configuration represents a traditional always-on deployment in which the inference server remains continuously active and the model is persistently loaded into GPU memory. This setup serves as a performance upper bound and a reference point for resource utilization.

Metric	Value
Startup time	81.0 sec
First request latency	0.803 sec
Warm latency (mean)	0.0199 sec
Warm latency (median)	0.0196 sec
Throughput (concurrency 4)	106.68 req/s
Throughput (concurrency 8)	203.87 req/s
Idle GPU memory	20972 MiB
Idle power draw	~28 W

Table 1: Baseline (always-on) performance and resource usage

The baseline system exhibits highly stable and predictable performance once the server is initialized. Warm latency remains consistently low (approximately 0.02 seconds), and throughput scales effectively with increased concurrency. These results confirm that keeping the model continuously resident in GPU memory allows the system to achieve near-optimal inference performance.

However, this performance comes at a significant cost. Even in the absence of requests, GPU memory remains fully allocated (approximately 21 GB), and power consumption persists at around 28 W. This indicates that the system continuously consumes computational resources regardless of workload, leading to substantial inefficiency during idle periods.

4.2 Naive Serverless

The naive serverless configuration removes persistent infrastructure by launching the inference server only when a request arrives and terminating it immediately afterward. This approach aims to eliminate idle resource usage.

Metric	Value
Startup time	78.0 / 42.0 / 34.0 sec
Startup time (avg)	~51.3 sec
First request latency	0.776 / 0.791 / 0.054 sec
First request latency (avg)	~0.540 sec
Idle GPU memory	~0 MiB
Idle power draw	~0–5 W

Table 2: Naive serverless performance and resource usage

The naive serverless approach successfully eliminates idle resource consumption. GPU memory usage drops to near zero when the server is not running, and power consumption is significantly reduced. This demonstrates that serverless deployment can effectively improve resource efficiency under low-demand conditions.

However, this efficiency introduces a substantial performance penalty. Startup time dominates the overall response latency, often exceeding tens of seconds. The first request latency is significantly higher than in the baseline system, reflecting the cost of loading the model and initializing the inference engine.

An important observation is that startup time decreases across repeated runs. This behavior is attributed to caching effects, including locally stored model weights and reused compilation artifacts. As a result, later runs do not represent fully cold starts, highlighting that cold-start latency can vary depending on system state and prior executions.

Overall, naive serverless presents an extreme trade-off: it minimizes idle resource usage but significantly degrades responsiveness due to cold-start overhead.

4.3 Keep-warm Serverless

The keep-warm configuration introduces a simple optimization strategy by periodically sending lightweight requests to prevent the server from shutting down. This approach aims to reduce cold-start latency while maintaining some level of resource efficiency.

Metric	Value
Startup time	80.0 sec
First request latency (kept-warm)	0.411 sec
Warm latency (mean)	0.0199 sec
Warm latency (median)	0.0196 sec
Throughput (concurrency 4)	111.80 req/s
Idle GPU memory	~16051 MiB
Idle power draw	~28 W

Table 3: Keep-warm serverless performance and resource usage

The keep-warm strategy does not reduce the initial startup cost, as the server still requires a full initialization phase. However, once the system is kept active, subsequent requests experience significantly improved latency. The kept-warm first request latency is reduced compared to naive serverless and approaches the baseline performance.

Warm latency and throughput closely match the always-on baseline, indicating that the system effectively operates in a warm state. This confirms that periodic requests successfully prevent the model from being unloaded or reinitialized.

Despite these performance improvements, resource usage increases compared to naive serverless. GPU memory remains partially allocated during idle periods, and power consumption remains comparable to the baseline system. This shows that keep-warm reduces latency at the cost of reintroducing persistent resource consumption.

4.4 Comparison and Discussion

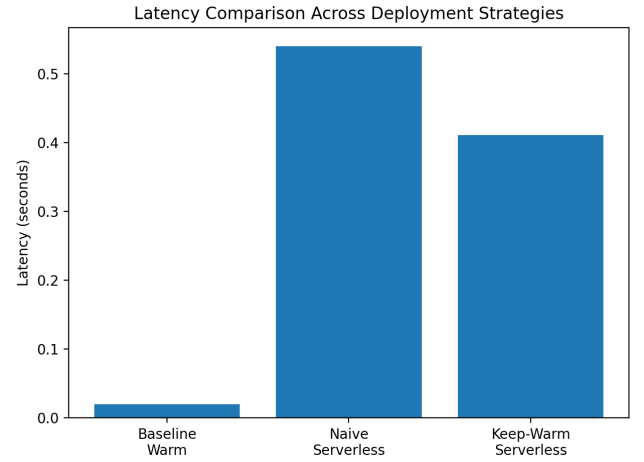


Figure 2: Latency comparison across deployment strategies. Naive serverless shows higher latency, while keep-warm recovers performance close to the baseline warm state.

The results clearly illustrate a fundamental trade-off between performance and resource efficiency. The always-on baseline provides the best performance, achieving consistently low latency and high throughput. However, it incurs continuous resource usage, even when idle.

In contrast, naive serverless eliminates idle resource consumption but introduces significant cold-start latency, making it unsuitable for latency-sensitive applications.

The keep-warm strategy represents a middle ground. By preventing the system from returning to a cold state, it achieves performance comparable to the baseline while partially sacrificing resource efficiency. This demonstrates that simple mechanisms can mitigate cold-start overhead, but they inherently reintroduce persistent resource usage.

Metric	Baseline	Naive Serverless	Keep-warm Serverless
Startup time	81.0 sec	~51.3 sec	80.0 sec
First request latency	0.803 sec	~0.540 sec	0.411 sec
Warm latency	0.0199 sec	N/A	0.0199 sec
Throughput (concurrency 4)	106.68 req/s	N/A	111.80 req/s
Idle GPU memory	20972 MiB	~0 MiB	~16051 MiB
Idle power draw	~28 W	~0–5 W	~28 W

Table 4: Comparison of deployment strategies

These findings highlight a key limitation of serverless inference systems: reducing cold-start latency typically requires maintaining model residency in GPU memory. As a result, achieving both low latency and high resource efficiency simultaneously remains a challenging problem.

5 Conclusion

Three deployment models are assessed in this paper for large language model (LLM) inference: an always-on baseline, naive serverless, and keep-warm serverless.

As shown by the results, there is a trade-off between performance and efficiency when using LLMs in the cloud. The always-on baseline demonstrates the highest performance in terms of latency and throughput; however, its downside is continuous GPU resource consumption, regardless of activity level. Naive serverless offers better resource efficiency by avoiding unnecessary GPU usage during idle periods; however, it suffers from the cold-start problem due to high initialization costs. The keep-warm approach significantly reduces cold-start latency, but at the expense of increased GPU resource usage.

It can be concluded that straightforward serverless solutions cannot effectively balance cold-start latency and resource utilization. Reducing cold-start time requires keeping model states in GPU memory to enable faster startup.

Further improvements should focus on more advanced system-level optimization techniques, such as partial model loading or multi-tier checkpoint storage. GPU sharing across workloads and adaptive job scheduling may also be effective.

Acknowledgments

The author would like to thank Professor Haiying Shen for guidance and support throughout this project. The author also thanks Shutian Luo for valuable assistance and advice during the experimental process.

References

- [1] Zinuo Cai et al. 2024. SMSs: Stateful Model Serving in Metaverse with Serverless Computing and GPU Sharing. *IEEE Journal on Selected Areas in Communications* 42, 3 (2024), 799–811. doi:10.1109/JSAC.2024.3362664
- [2] Yao Fu et al. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. *arXiv preprint arXiv:2401.00000* (2024).
- [3] Seoyoung Kim et al. 2025. Oltunes: Online Learning-Based Auto-Tuning System for Deep Learning Inference in Heterogeneous GPU Clusters. *Cluster Computing* (2025).
- [4] Zijun Li et al. 2025. Lightweight and Holistic-Scalable Serverless Secure Container Runtime for High-Density Deployment and High-Concurrency Startup. *IEEE Trans. Comput.* 74, 8 (2025), 2621–2634. doi:10.1109/TC.2025.3543904

- [5] Pengwei Wang et al. 2025. Optimizing Serverless Performance through Game Theory and Efficient Resource Scheduling. *IEEE Trans. Comput.* 74, 6 (2025), 1990–2002. doi:10.1109/TC.2025.3534768